Функції, що повертають результат, і 595 каскадом

Урок 3 — 2MAK, SST-INF-01c — `return`, функція `invia595`, масиви, два і чотири 595 підряд

29/10/2026

Примітка

На цьому уроці ми завершуємо тему **функцій**: розглядаємо функцію, яка **отримує числа** і **повертає результат** (гіпотенузу прямокутного трикутника). Потім повертаємося до програми для **74HC595** з роздаткового матеріалу A02 і **реорганізуємо** її: цикл, що відправляє 8 бітів, стає функцією **`invia595`**, числа для показу потрапляють у **масив** (*array*), а наприкінці ми з'єднуємо **два 595 каскадом** (*in cascata*, 16 світлодіодів) і, як виклик, **чотири** (32 світлодіоди), використовуючи завжди лише **три дроти** Arduino.

Увага

Контрольну перенесено. Контрольну з firmware, заплановану на сьогодні, 29 жовтня, **скасовано** і **перенесено на четвер, 5 листопада**. Вона охоплює **усю** програму: вміти під'єднати до Arduino **світлодіоди і кнопки**, надіслати дані в **монітор порту** (*monitor seriale*, як минулого року), а також те, що ми зробили цього року: **функції** і запис у **595**.

1 Повторення: що таке функція

Функція — це **набір команд**, такий собі **міні-програма**, яку можна викликати з головної програми. Приклад, який ми добре знаємо, — **595**: щоразу, коли ми кладемо дані в 595, треба дати **тактовий імпульс** (*impulso di clock*) на вивід SRCLK. Імпульс робиться **4 рядками** коду: `digitalWrite` високий рівень на дроті тактування, `delay` для очікування, `digitalWrite` низький рівень і ще невеликий час для надійності.

Замість того щоб щоразу переписувати ці 4 рядки, ми зібрали їх у функцію **`carica`** ("завантаж"; ім'я вибирає той, хто програмує: "carica" здавалося відповідним до того, що вона робить).

```
void carica()
{
  digitalWrite(SRCLK, HIGH);
  delay(1);
  digitalWrite(SRCLK, LOW);
  delay(1);
}
```

У `loop` досить написати **`carica();`**: Arduino виконує цю маленьку програму і, коли закінчить, **повертається** до того, що робив.

1.1 Тип значення, що повертається: приклад із бутербродом

Функція може **приймати** **параметри**, тобто дані, які ми їй передаємо, щоб **налаштувати** її роботу, і може **повертати** **результат**. Загальна форма така:

```
tipo NomeFunzione(tipo parametro1, tipo parametro2)
{
  ...
}
```

(`tipo` = тип, `NomeFunzione` = ім'я функції, `parametro` = параметр.) Ім'я вибираємо ми; **перед** **ім'ям** пишуть **тип** **даних,** **що** **повертаються** (*tipo di ritorno*), тобто тип результату, який функція нам віддає, коли закінчить.

Приклад

Друг, який перетворює гроші на їжу. Уявіть, що ви "програмуєте" друга, щоб він **перетворював** **гроші** **на** **їжу**: я даю йому гроші, а він (не знаю як, і мене це не цікавить: може, іде у фастфуд і купує бутерброд) **повертає** **мені** **їжу**. Найкраще те, що коли я його вже "запрограмував", я про це більше не думаю: даю гроші й отримую бутерброд. Гроші — це **параметр**, бутерброд — **значення,** **що** **повертається**.

А якщо друг бере гроші і **тікає**, сам з'їдає бутерброд і нічого мені не приносить? Такий тип функції теж існує: це функція **`void`**. `void` означає, що функція **нічого** **не** **повертає**: це тип "без розміру". Наша `carica()` саме така: вона нічого не отримує і нічого не повертає, лише піднімає й опускає рівень на виводі.

Типи результату, які ми знаємо, — ті самі, що й у змінних: **`int`**, **`char`**, **`float`**, а також **`void`** для "жодного результату". Бувають функції, яким ми даємо **цілі** **числа**, а вони повертають результат **іншого** **типу**, наприклад `float`: усе залежить від того, що вони мають робити.

2 Функція з результатом: гіпотенуза

2.1 Навіщо це потрібно: гра Pong

Приклад, який трапляється часто. Треба зробити гру **Pong** на **світлодіодній** **матриці**, тобто **сітці** світлодіодів, які ми вмикаємо залежно від того, що має відбуватися. Якщо світиться один світлодіод і м'ячик переміщується, треба ввімкнути світлодіод, який, наприклад, на **4** **клітинки** **правіше** і на **3**

клітинки вище. Щоб обчислити швидкість, з якою "вистрілити" м'ячик, нам потрібна відстань між двома світлодіодами: ми знаємо зміщення по горизонталі і по вертикалі, а відстань обчислюється за теоремою Піфагора.

Визначення

Теорема Піфагора. У прямокутному трикутнику з катетами a і b гіпотенуза (*ipotenusa*) c дорівнює

$$c = \sqrt{a^2 + b^2}$$

тобто квадрат, побудований на гіпотенузі, дорівнює сумі квадратів, побудованих на катетах.

2.2 Які типи використати

Створюємо функцію `ipotenusa`, якій ми передаємо два катети і яка нам відповідає гіпотенузою.

- **Параметри:** якщо катети — це відстані між пікселями (або світлодіодами), вони є цілими числами: половини пікселя не існує, не можна ввімкнути піксель 3,5. Це як попросити когось сісти посередині між двома робочими місцями: він сяде або на перше, або на друге. Тож параметри — це два `int`: `int a, int b`.
- **Результат:** з квадратним коренем навряд чи вийде ціле число, тому результат буде `float`, число з комою.

```
float ipotenusa(int a, int b)
{
  int somma = a * a + b * b;
  float ip = sqrt(somma);
  return ip;
}
```

Спочатку обчислюємо вираз під коренем, `a * a + b * b`: з двома цілими числами він усе одно виходить цілим, тому ми кладемо його в змінну `int somma` (*somma* = "сума"). Потім обчислюємо квадратний корінь функцією `sqrt` (з англійської *square root*, "квадратний корінь"): `sqrt` — теж функція, уже готова, яка отримує число і повертає його корінь. Результат іде у `float ip`, і за допомогою `return ip;` ми повертаємо його тому, хто викликав функцію.

2.3 Використання функції в `loop`

```
void loop()
{
  float x = ipotenusa(3, 4);
  float z = ipotenusa(9, 12);
  float y = ipotenusa(4, 7);
}
```

Що відбувається, коли Arduino доходить до `ipotenusa(3, 4)`?

1. викликає функцію, тобто виконує її, і передає їй два числа: 3 потрапляє в `a`, 4 потрапляє в `b`;
2. функція обчислює $3 \cdot 3 = 9$, $4 \cdot 4 = 16$, $9 + 16 = 25$;

3. обчислює $\sqrt{25} = 5$ і **віддає назад** 5;
4. 5 **потрапляє в** `x`: `x` дорівнює 5, це результат обчислення, зробленого функцією.

З `ipotenusa(9, 12)` 9 іде в `a`, а 12 — у `b`: $9 \cdot 9 = 81$, $12 \cdot 12 = 144$, $81 + 144 = 225$, $\sqrt{225} = 15$, отже `z` дорівнює 15. З `ipotenusa(4, 7)`: $16 + 49 = 65$ і $\sqrt{65} \approx 8{,}06$, отже `y` дорівнює приблизно 8,06: тут кома справді потрібна. (3, 4, 5 і 9, 12, 15 — дві "щасливі трійки", в яких корінь виходить цілим; зазвичай так не буває.)

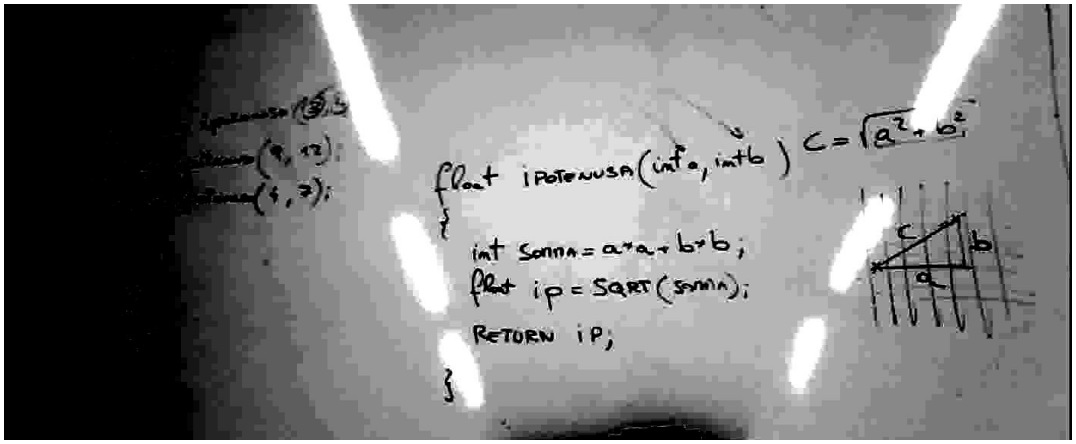


Рис. 1: Дошка: ліворуч `loop` із трьома викликами і розрахунок $81 + 144 = 225$; посередині функція `float ipotenusa(int a, int b)` із `somma`, `sqrt` і `return`; праворуч трикутник із катетами a, b і гіпотенузою c.

Увага

Результат треба покласти в змінну правильного типу. Якщо `ipotenusa` повертає `float`, не варто писати `int z = ipotenusa(9, 12);`: **`int` може помістити у `float`**, але **`float` не може помістити в `int`**, нічого не втративши. Якщо покласти число з комою в ціле, **втрачається вся дробова частина**: 8,06 стало б 8.

2.4 Чому це вигідно

Змінні `x`, `z`, `y` — це **контейнери**, в які ми кладемо результати функції, яку **використовуємо кілька разів**: формула завжди та сама, але **якщо змінити вхідні дані**, змінюється результат. Без функції для кожної гіпотенузи довелося б переписувати дві інструкції (суму і корінь): шість рядків замість трьох. А тут функція маленька: уявіть обчислення з **40 рядків** складних розрахунків, яке треба щоразу повторювати. З функцією ми **заощаджуємо** багато написаного коду і **зменшуємо ймовірність помилки**: що менше я пишу, то менша ймовірність помилитися.

Нам важливо зрозуміти ось що: функції **можна передати значення**, і функція віддає мені **результат, який залежить** від значень, що я їй дав.

3 Повторення: побітове І та зсув

Повернімося до програми для 595 з минулого разу (матеріал A02). Наприкінці уроку ми бачили останню новинку: використовуючи **побітове І** (*AND bit a bit*, `&`) і **зсув** (*scorrimento*, `<<`), ми могли

“вистрілювати” в 595 **по одному** біти, з яких складається число. Як приклад ми вибрали **181**, але це може бути будь-яке число.

Примітка

Роздаткові матеріали треба читати і вивчати. Перш ніж пояснювати цю частину ще раз, ми в класі взяли пів години, щоб **відкрити і прочитати** матеріал A02, де все це вже пояснено крок за кроком. Роздатковий матеріал не “читають, і все”: розділ, частину книжки **перечитують кілька разів**, поки справді не зрозуміють, про що йдеться. “Я прочитав, але не пам’ятаю” означає, що матеріал не зрозумілий: саме з цього і починається навчання.

```
for (int i = 0; i < 8; i++)
{
  digitalWrite(SER, 181 & (B00000001 << i));
  delay(1);
  carica();
}
```

3.1 Цикл `for`

`for (int i = 0; i < 8; i++)` **повторює** **8** **разів** те, що стоїть між фігурними дужками, і при кожному повторенні змінна `i` має **інше** **значення**: першого разу 0, потім 1, 2, 3, на п'ятому колі 4, потім 5, 6 і 7. Коли вона дорівнює 7 і цикл закінчено, він повертається нагору, **збільшує** `i`, яка стає 8: 8 **не** **менше** **за** **8**, і цикл завершується. Разом 8 повторень, і `i` змінюється **від** **0** **до** **7**.

3.2 Що записує `digitalWrite` на кожному колі

На кожному колі ми записуємо на вивід **SER** (`#define` з минулого разу, який до компіляції стає числом **3**) високий або низький рівень, що залежить від результату `181 & (B00000001 << i)`. `B00000001` — це число 1, записане у двійковій системі; `<< i` **зсуває** **його** **вліво** на `i` позицій.

Щоб зрозуміти результат І, треба записати 181 у двійковій системі: **181** **=** **10110101**. Під ним записуємо зсунуте число і виконуємо І **стовпчик** **за** **стовпчиком**: $1 \cdot 1 = 1$, а якщо є хоча б один 0, результат 0.

`i`	`1 << i` (двійкове)	`181 & ...`	результат	на виводі
0	00000001	181 & 1	1	високий (H)
1	00000010	181 & 2	0	низький (L)
2	00000100	181 & 4	4	високий (H)
3	00001000	181 & 8	0	низький (L)
4	00010000	181 & 16	16	високий (H)
5	00100000	181 & 32	32	високий (H)
6	01000000	181 & 64	0	низький (L)
7	10000000	181 & 128	128	високий (H)

Визначення

Істина і хибність для комп'ютера. **Лише** **нуль** **—** **хибність**; **будь-яке** **інше** **число** **—** **істина**. Тож 1, 4, 16, 32, 128 — усі "істинні", і `digitalWrite` ставить на виводі **високий** рівень (це те саме, що написати `HIGH`); 0 — "хибність", і на виводі **низький** рівень.

Після кожного `digitalWrite` ми трохи чекаємо і викликаємо `carica()`, яка піднімає й опускає SRCLK і вводить цей біт у 595. Так перше коло завантажує високий рівень, друге — низький, третє — високий і так далі: **трьома** **рядками** **коду** ми викинули все число 181. Якщо потрібне інше число, досить **змінити** **181**.

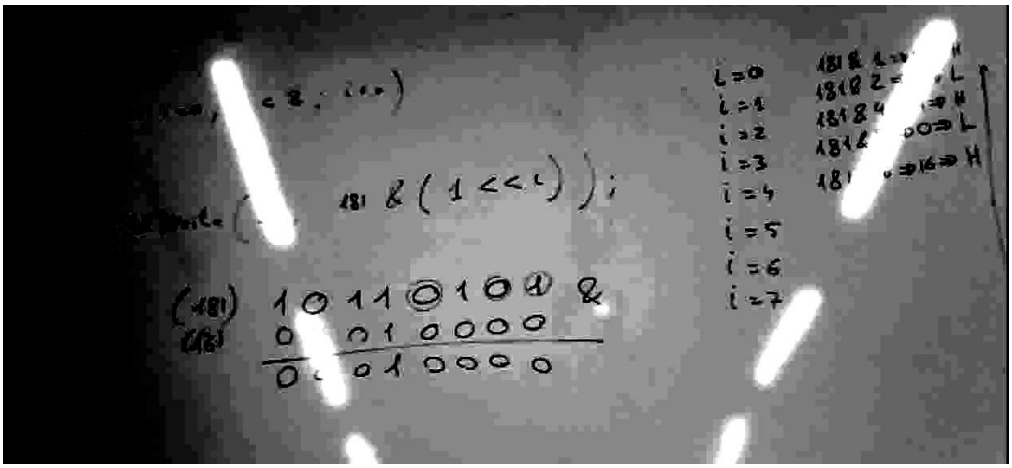


Рис. 2: Дошка: `for`, рядок `digitalWrite(3, 181 & (1 << i))`, 181 у двійковій системі (10110101), поєднане через I з `1 << 4` = 00010000, а праворуч значення `i` з результатами: $181\&1 = 1 \Rightarrow$ H, $181\&2 = 0 \Rightarrow$ L, $181\&4 = 4 \Rightarrow$ H, $181\&8 = 0 \Rightarrow$ L, $181\&16 = 16 \Rightarrow$ H.

3.3 Скільки часу це займає

Кожен біт потребує `delay(1)` плюс два `delay(1)` всередині `carica()`: **3 мілісекунди на біт**, отже $3 \cdot 8 = 24$ мс на весь байт, плюс 2 мс імпульсу на RCLK: разом приблизно **26 мілісекунд**. Для нас це практично **миттєво**.

3.4 Інші числа: 15, 192, 255

Якщо замість 181 поставити **15**, що у двійковій системі **00001111**, світяться лише **4 світлодіоди з одного боку**. Щоб дізнатися, як виглядає число у двійковій системі, можна скористатися **калькулятором** комп'ютера або порахувати вручну: **ділимо на 2**, беремо **остачі**, а потім читаємо їх **у зворотному порядку**. З **255** (11111111) світяться **усі**: з резисторами на **470 Ω** 595 витримує; з резисторами на 220 Ω загальний струм був би завеликим (ми бачили це в A02).

Ніхто не забороняє писати число **прямо у двійковій системі**, якщо так зручніше: `B01010101` означає "світиться, вимкнено, світиться, вимкнено…" і читається одразу.

4 Функція `invia595`

Наступний природний крок — взяти цей `for` і **згрупувати його у функцію**, щоб **прибрати з `loop`** шматок коду, який стосується лише деталей 595. Замість 181 ми хотіли б мати можливість поставити **змінну**, наприклад `char d = 181;`, яку змінюємо під час програми залежно від того, що нам потрібно. Тоді число стає **параметром** функції: "гроші", які ми даємо, щоб отримати "бутерброд". (`invia595` = "надішли в 595".)

```
void invia595(char d)
{
  for (int i = 0; i < 8; i++)
  {
    digitalWrite(SER, d & (B00000001 << i));
    delay(1);
    carica();
  }
  // вмикаю RCLK, щоб перенести дані з регістра на виходи
  digitalWrite(RCLK, HIGH);
  delay(1);
  digitalWrite(RCLK, LOW);
  delay(1);
}
```

for **переписано** **без** **змін**: де було 181, тепер стоїть **d**. Рядки **RCLK** (імпульс, який виводить дані на виходи) ми теж переносимо у функцію, бо вони є частиною надсилання. Відтепер **як** **влаштований** **595, нас** **більше** **не** **цікавить**: у loop досить сказати **invia595(число)**.

```
void loop()
{
  invia595(181);
  delay(500);
  invia595(42);
  delay(500);
  invia595(255);
  delay(500);
  invia595(B01010101);
  delay(500);
  invia595(B10101010);
  delay(500);
}
```

595 починає **анімувати** **послідовність**: 181, 42, усі світяться (255), непарні, парні. З 500 мс це трохи швидко: можна поставити delay на **1000**, щоб сповільнити. І Arduino використовує **три** **дроти**, а не вісім.

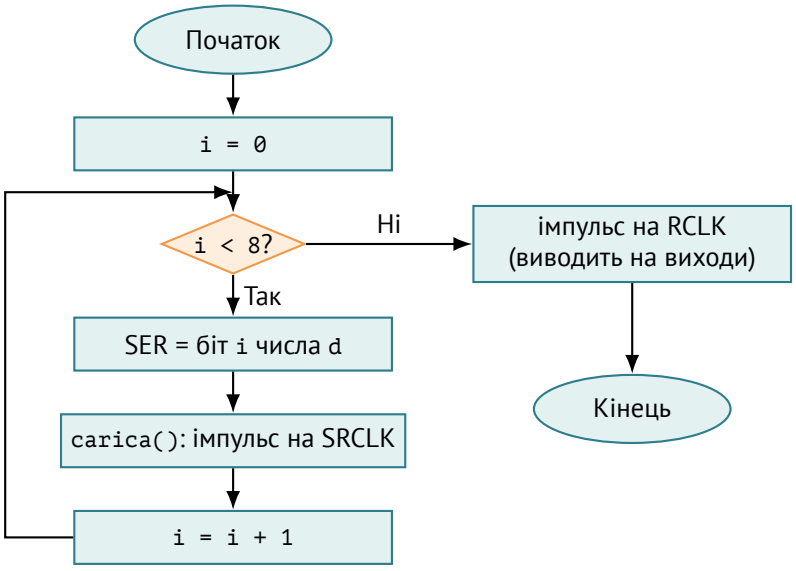


Рис. 3: Блок-схема функції invia595(d).

Примітка

Тип `char` і числа понад 127. У цьому курсі `char` використовують зі значеннями від 0 до 255. У стандартному C, зокрема на Arduino, `char` є знаковим ($-128 \ldots +127$), а 0–255 — це діапазон `unsigned char` (або `byte`): значення на кшталт 181 зберігається з іншим знаком, але **8 бітів ті самі**, тож побітове I і 595 усе одно працюють. Хто хоче бути точним, може писати `byte d` замість `char d`.

5 Масив для послідовності

Пам'ятаєте **масиви** (*array*) з минулого року? Масив — це структура, яка дає змогу **зібрати кілька чисел в одному об'єкті**. Створюємо масив із **5 елементів** з числами послідовності (*sequenza*) і замінюємо всі виклики в `loop` одним `for`:

```
char sequenza[5] = {181, 42, 255, B01010101, B10101010};

void loop()
{
  for (int i = 0; i < 5; i++)
  {
    invia595(sequenza[i]);
    delay(500);
  }
}
```

Результат **той самий**: викидає 181, чекає, потім 42, чекає, потім 255 і так далі (з `delay` на **1000** це краще видно). Але тепер, якщо завтра я вирішу поставити **50 послідовностей**, я додам 50 чисел у масив (і зміню 5 у `for`) **не пишучи жодного рядка коду більше**, лише дані, які треба викинути. На уроці масив справді подовжили до 50 чисел:

```
char sequenza[50] = {181, 42, 255, B01010101, B10101010,
                     B00000000, B11111111, B00001111, B11110000,
                     ... altri numeri, fino a 50 ...};

void loop()
{
  for (int i = 0; i < 50; i++)
  {
    invia595(sequenza[i]);
    delay(1000);
  }
}
```

(`altri numeri, fino a 50` = “інші числа, до 50”.) 595 викидає їх **усі, по одному**, послідовно. Так створюється **світлова гра**, яка вам до вподоби.

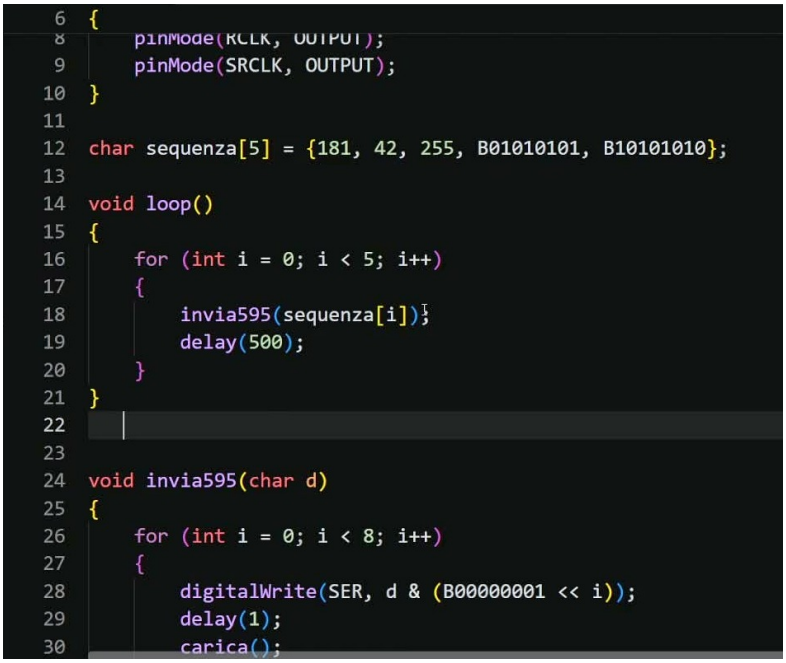


Рис. 4: Код у Visual Studio Code: масив `sequenza[5]`, `loop` із `for` від 0 до 4 і початок функції `invia595(char d)`.

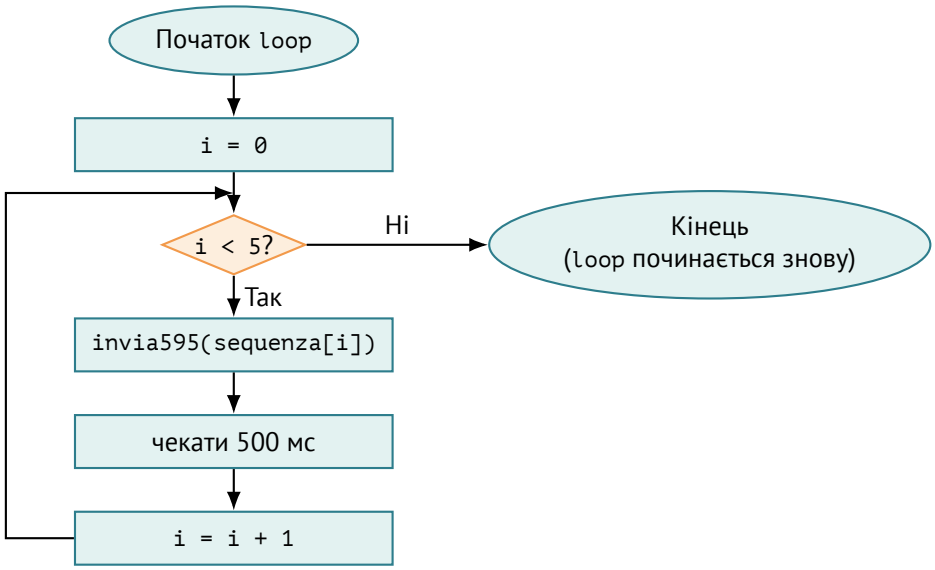


Рис. 5: Блок-схема `loop` з масивом: `i` вказує позицію в масиві, `sequenza[i]` — число, що стоїть на цій позиції.

6 Кілька 595 каскадом

Тут ми керуємо лише одним 595; **Pong**, який є в іншій лабораторії, має їх **набагато більше** (близько 128), поставлених **один за одним**: це називається **каскадом** (*in cascata*). З'єднання такі:

- дроти **RCLK** (вивід 12) і **SRCLK** (вивід 11) під'єднуються **паралельно** до всіх 595: 12 одного йде до 12 іншого, 11 — до 11;
- а дріт даних проходить **послідовно**: від Arduino він **входить у вивід 14** (SER) першого 595, **виходить з виводу 9** (Q7', послідовний вихід) і входить у 14 наступного 595, і так далі.

Так можна під'єднати **скільки завгодно** мікросхем, завжди **трьома дротами** Arduino.

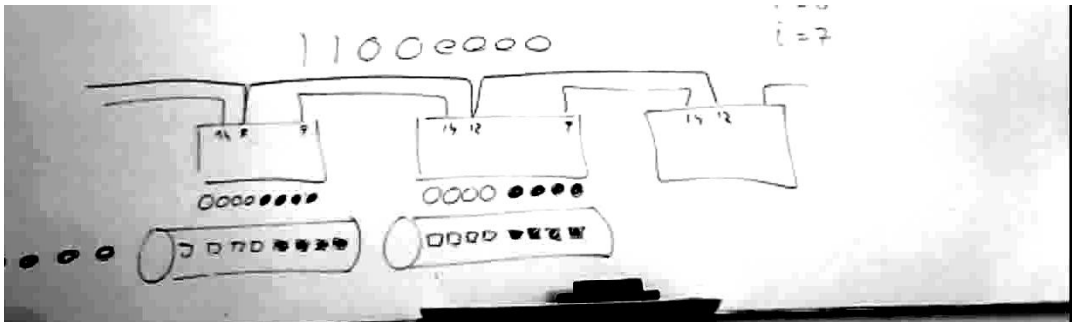


Рис. 6: Дошка: три 595 каскадом (угорі виводи 14, 12, 11 і 9: 9 одного йде до 14 наступного, 12 і 11 спільні), а внизу "трубки" з кульками: коли входять ще 8 кульок, перші 8 виштовхуються в наступну трубку.

6.1 Монтаж другого 595 у Tinkercad

Схему розширюють, **копіюючи весь** блок 595 + 8 світлодіодів + резистори на другу макетну плату і роблячи **з'єднання між однією схемою та іншою**: вивід 9 першого до 14 другого, 11 з 11, 12 з

12, і, звісно, **живлення** (5 В і GND) теж до другої макетної плати.

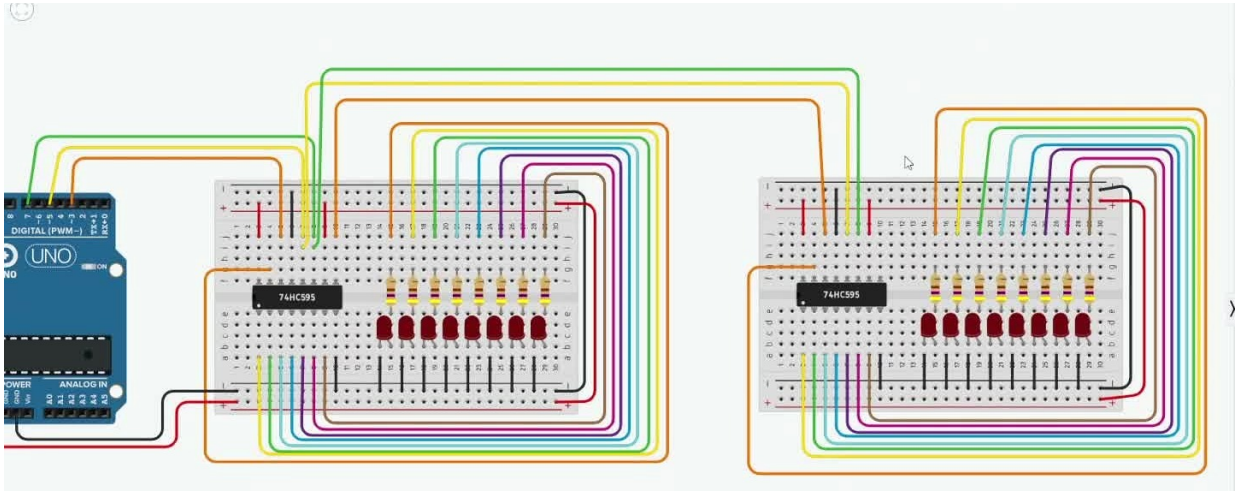


Рис. 7: Два 74HC595 каскадом у Tinkercad, кожен із 8 світлодіодами: Arduino з'єднаний лише з першим; другий отримує дані з виводу 9 першого і два тактові сигнали паралельно.

Увага

Не забувайте про живлення. Під час першої спроби другий 595 не працював: усю увагу приділили дротам між двома мікросхемами, а **живлення** **не** **підвели** до другої макетної плати. У Tinkercad мікросхема без живлення не "вибухає", вона просто **не** **працює**.

6.2 Що відбувається з двома 595: дві трубки

Уявімо два 595 як **дві** **трубки,** **з'єднані** **послідовно**, кожна з 8 віконцями. Вкидаємо 8 кульок (4 червоні й 4 білі): після перших 8 бітів кульки **заповнюють** **першу** **трубку**. Якщо тепер вкинути **ще** **8** **кульок**, вони **проштовхують** **уперед** перші, які розподіляються в **другій** **трубці**, а нові займають їхнє місце в першій.

Отже:

- **перший** надісланий байт зрештою опиняється в **другому** 595 (найдальшому від Arduino);
- **останній** надісланий байт лишається в **першому** 595.

Якщо надсилати **завжди** **одне** **й** **те** **саме** **число**, наприклад 15, ми бачимо ту саму послідовність (вимкнено $\times 4$, світиться $\times 4$) на **обох**: `loop` повторюється, і друге надсилання проштовхує перше в другий 595.

З двома різними числами це добре видно: надсилаємо спочатку **224** (11100000, три світлодіоди світяться і п'ять вимкнено), потім **15** (00001111). 224 входить у перший 595, потім надходить 15 і **проштовхує** **його** **в** **другий**: у кінці 15 стоїть у **лівому** 595 (першому, біля Arduino), а 224 — у **правому**.

6.3 “Мерехтіння” і де ставити RCLK

Коли імпульс на RCLK стоїть усередині `invia595`, світлодіоди **блимають**: видно **темні спалахи**. Це називається **мерехтінням** (*flickering*) і є невеликою **помилкою програмування**: щоразу, коли ми вкидаємо число, ми одразу **виводимо** його на виходи. Спочатку виводимо після 224 (яке на мить з’являється в першому 595), потім знову після 15. **Виведення** (*pubblicazione*) треба робити **лише один раз**, **після того**, як вкинули **усі** дані.

Рішення — **прибрати імпульс на RCLK** з `invia595` і поставити його в `loop`, **після останнього надсилання**: тепер світлодіоди більше не блимають і світять **рівно і стабільно**.

6.4 Одна функція для обох тактових сигналів: `carica(pin)`

Дивлячись на код, помічаємо одну річ: послідовність `digitalWrite` високий, `delay`, `digitalWrite` низький, `delay` **однакова** для SRCLK і для RCLK. Змінюється **лише назва виводу** (7 для SRCLK, 5 для RCLK). Тут виводів два, але уявіть, що їх 18 і всі треба вмикати й вимикати: ніхто не пише 18 разів одне й те саме. Тому ми **стандартизуємо**: робимо функцію `carica` **налаштовуваною**, додаючи **параметр** — **вивід**, на який треба дати імпульс.

```
void carica(char pin)
{
  digitalWrite(pin, HIGH);
  delay(1);
  digitalWrite(pin, LOW);
  delay(1);
}
```

Замість `SRCLK` усередині функції стоїть слово **`pin`**, а коли ми її викликаємо, кажемо, який вивід використати: **`carica(SRCLK)`** на кожному біті, **`carica(RCLK)`** один раз у кінці, щоб вивести дані. Ця зміна **не змінює роботу програми**: ми лише винесли з функції інформацію про вивід, яка раніше була записана всередині. Натомість ми **скоротили код ще на три рядки**: тепер він написаний “по-серйозному”, жоден програміст не зробив би коротше. Ми дійшли до цього крок за кроком.

6.5 Повна програма для двох 595

Це програма, опублікована в Classroom (файл `doppio595`, “подвійний 595”). Щоб перевірити, що у вас оновлена версія: у `loop` має бути **`carica(RCLK)`**.

```
#define SER 3
#define RCLK 5
#define SRCLK 7

void setup()
{
  pinMode(SER, OUTPUT);
  pinMode(RCLK, OUTPUT);
  pinMode(SRCLK, OUTPUT);
}

void loop()
{
  invia595(224);
  invia595(15);
  // вмикаю RCLK, щоб перенести дані з регістра на виходи
  carica(RCLK);
  delay(500);
}

void invia595(char d)
{
  for (int i = 0; i < 8; i++)
  {
    digitalWrite(SER, d & (B00000001 << i));
    delay(1);
    carica(SRCLK);
  }
}

void carica(char pin)
{
  digitalWrite(pin, HIGH);
  delay(1);
  digitalWrite(pin, LOW);
  delay(1);
}
```

Увага

Типові помилки, помічені на робочих місцях.

- Старе число, що лишилося в коді (наприклад, забуте `181` замість `d` усередині `invia595`): функція має використовувати **параметр** `d`, у який копіюється передане число (224, 15…).
- `void carica()` **без** `char pin` у дужках, тоді як в іншій частині коду викликається `carica(SRCLK)`: якщо функція не оголошує параметр, програма не компілюється.
- Дроти, які на малюнку **проходять над** рядом макетної плати, але **під'єднані** там, де не повинні: дріт, покладений на отвір, під'єднаний. Якщо на схемі його немає, його треба видалити.

Найшвидший спосіб перевірити: **скопіювати весь** код, опублікований у Classroom (Ctrl+A, Ctrl+C), вставити його замість свого і запустити симуляцію. Якщо так працює, помилка була в коді; якщо ні — у схемі.

Приклад

Світлодіоди “майже вимкнені”: шукаємо несправність осцилографом. Якщо деякі світлодіоди ледь світяться або поводяться дивно, у Tinkercad можна скористатися **осцилографом**: чорний дріт до **мінуса**, червоний — на **вихід**, який перевіряємо. Увага до масштабу: при **10 мкВ на поділку** сигнал 22 мкВ здається великим, але насправді це **нуль** (невеликий струм дрейфу). Якщо ж показує приблизно **постійні 2 В**, вихід ні високий, ні низький: це рівень, який **погано вмикає** світлодіод. У такому випадку несправність була в схемі: у 595 **не було з’єднання з GND**. Після додавання дроту виходи знову дають 0 В або 5 В, і світлодіоди поводяться як очікувалося.

7 Виклик: 32 світлодіоди з чотирма 595

Остання частина уроку: керувати **32 світлодіодами**, тобто **чотирма 595** каскадом, і показати **чотири різні числа** на чотирьох групах по вісім, приблизно за 40 хвилин. Схему отримують, повторюючи каскадне з’єднання (9 → 14, спільні тактові сигнали, живлення до всіх макетних плат); у коді досить **надіслати чотири числа**, перш ніж виводити.

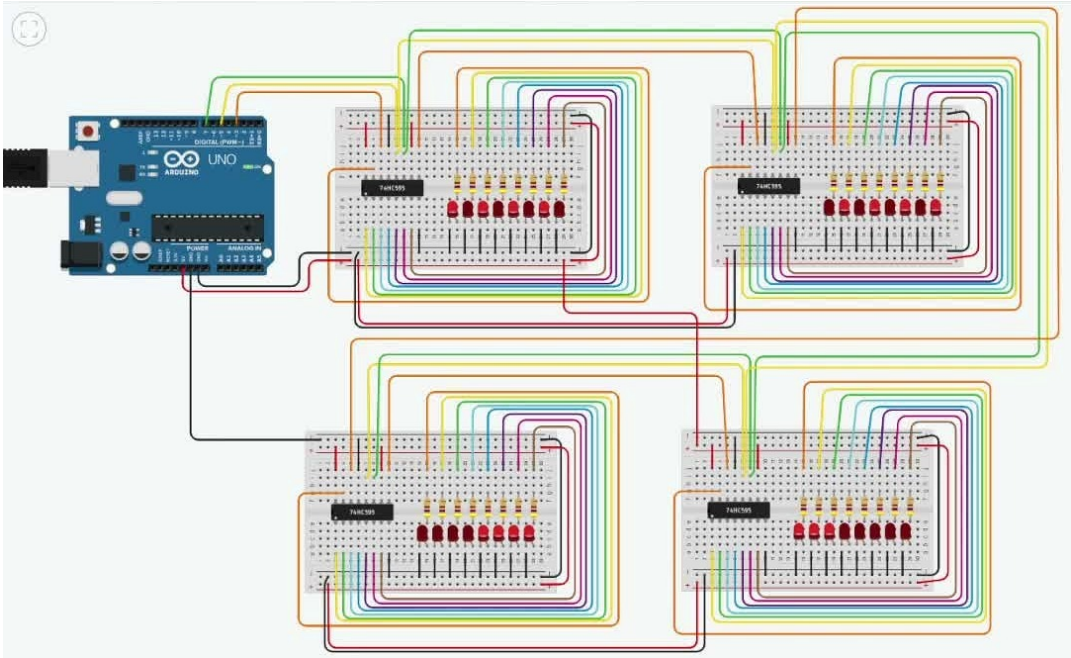


Рис. 8: Чотири 74HC595 каскадом у Tinkercad: 32 світлодіоди, якими керують лише три дроти Arduino.

```
void loop()
{
  invia595(224);
  invia595(15);
  invia595(0x55);
  invia595(0xAA);
  // вмикаю RCLK, щоб перенести дані з регістра на виходи
  carica(RCLK);
  delay(500);
}
```

`0x55` і `0xAA` — два числа, записані в **шістнадцятковій системі** (префікс **`0x`**): `0x55` = 85 = `B01010101` (світлодіод через один), `0xAA` = 170 = `B10101010` (протилежне чергування). Перше надіслане число (224) опиняється в **найдальшому** 595, останнє (0xAA) — у **першому**. Решта програми (`invia595` і `carica`) **така сама**, як для двох 595.

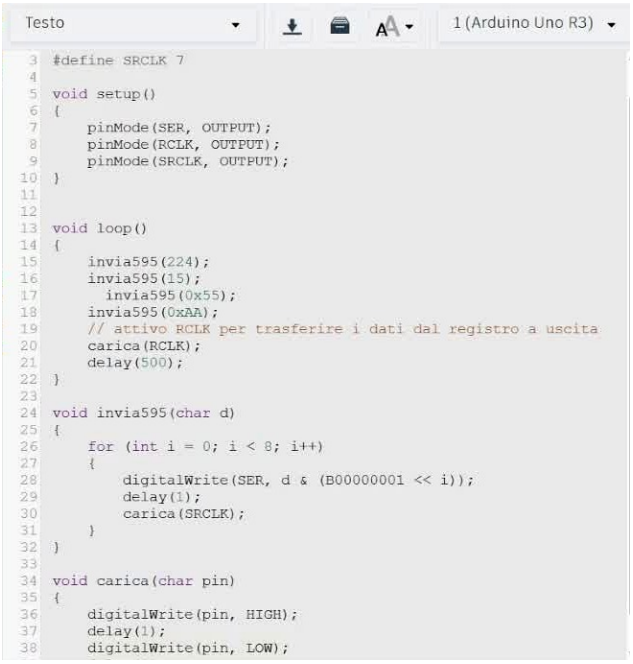


Рис. 9: Код рішення в редакторі Tinkercad: чотири `invia595`, а за ними лише один `carica(RCLK)`.

8 Що запам'ятати з цього уроку

- Функція має **тип значення, що повертається**, **ім'я** і **параметри**: `float ipotenusa(int a, int b)`. **`void`** означає, що вона **нічого не повертає**.
- Через **`return`** функція повертає результат, який треба покласти в змінну **правильного типу**: `float` в `int` втрачає дробову частину.
- **`sqrt`** обчислює квадратний корінь. $\sqrt{3^2+4^2}=5$, $\sqrt{9^2+12^2}=15$, $\sqrt{4^2+7^2}\approx 8{,}06$.
- Функції дають змогу **писати менше** і тому **менше помилятися**; якщо змінити вхідні дані, змінюється результат.
- Для комп'ютера **лише нуль — хибність**: `d & (B00000001 << i)` "істинне", якщо біт `i` числа `d` дорівнює 1. 181 = 10110101.
- **`invia595(d)`** відправляє 8 бітів `d`; **`carica(pin)`** дає імпульс на вивід, який ми їй передаємо (SRCLK на кожному біті, RCLK для виведення).
- **Масив** збирає кілька чисел в одному об'єкті: `sequenza[i]` — елемент на позиції `i` (від 0). Циклом `for` проходять усю послідовність.
- **595 каскадом**: вивід **9** першого до **14** наступного, **11** і **12** **паралельно**, живлення до всіх. **Перший** надісланий байт опиняється в **найдальшому** 595.
- Імпульс на **RCLK** дають **лише один раз, після всіх надсилань**: інакше світлодіоди **мерехтять**.

Приклад

До контрольної в четвер, 5 листопада: контрольна охоплює **увесь зміст роздаткових матеріалів** A01, A01bis, A02 і firmware минулого року (світлодіоди, кнопки, монітор порту, функції, 595).

Глосарій: італійські слова, які вживають на уроці

Italiano	**Українською**
funzione, parametro	функція, параметр
tipo di ritorno, restituire	тип значення, що повертається; повертати
panino, soldi, cibo	бутерброд, гроші, їжа
ipotenusa, cateto, radice quadrata	гіпотенуза, катет, квадратний корінь
somma	сума
numero intero / con la virgola	ціле число / число з комою
AND bit a bit, scorrimento	побітове І, зсув
vero / falso	істина / хибність
alto / basso	високий / низький (рівень)
invia, inviare	надішли, надсилати
carica, caricare	завантаж, завантажувати
array, sequenza	масив, послідовність
in cascata, in parallelo, in serie	каскадом, паралельно, послідовно
tubo, pallina	трубка, кулька
pubblicare (le uscite)	вивести (на виходи)
alimentazione	живлення
oscilloscopio	осцилограф
esadecimale	шістнадцятковий
verifica	контрольна